

Research Software Engineering in Python for Reproducible Computational Science

Sylwester Arabas

AGH University of Krakow

2026-04-09



AGH



previously on...

- ▶ what (or who) is **RSE**?
- ▶ Python as a glue language
- ▶ unit-aware coding/plotting (**Pint**)
- ▶ **reproducibility** in computational science
- ▶ automation & reuse (DRY)
- ▶ Python's "duck typing" (e.g. SciPy+Pint)
- ▶ JIT-compilation in Python (e.g. Numba)
- ▶ software vs. content licensing
- ▶ open-source: copyleft/permissive
- ▶ semantic versioning
- ▶ persistent archival @Zenodo (CERN)
- ▶ Python packaging (pyproject.toml)

plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- Definition of test-assignment and grading criteria



AGH



plan for today

- **Python projects: generating documentation (from docstrings)**
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- Definition of test-assignment and grading criteria



AGH



a Python function without a docstring

```
from random import random as u01
pi = lambda n: 4 * sum(abs(u01() - 1j * u01()) < 1 for _ in range(n)) / n
help(pi)
```

Help on function <lambda> in module __main__:

<lambda> lambda n



AGH



a Python function with a docstring

```
pi.__doc__ = "Estimate  $\pi$  using a Monte Carlo method.\n\nn: number of trials."
```

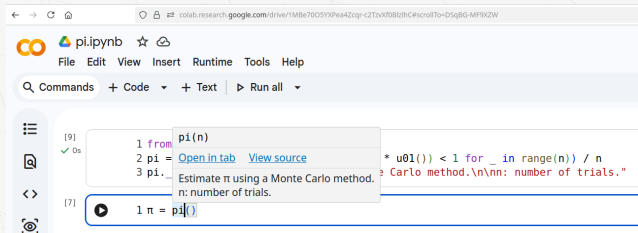
```
help(pi)
```

Help on function <lambda> in module __main__:

<lambda> lambda n

Estimate π using a Monte Carlo method.

n: number of trials.



The screenshot shows a Jupyter Notebook interface. The top bar indicates the file is named 'pi.ipynb'. The notebook contains two code cells. The first cell, labeled [9], defines a function `pi(n)` with a docstring: `Estimate  $\pi$  using a Monte Carlo method.` and `n: number of trials.`. A tooltip is visible over the function name `pi(n)` in the first cell, showing the function signature `pi(n)` and the docstring. The second cell, labeled [7], contains the code `1 pi = pi()`. The notebook interface includes a menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help, and a toolbar with icons for running, saving, and other actions.

<https://github.com/hgrecco/pint/blob/master/pint/registry.py> ↪ pint.readthedocs.io

```
165  def setup_matplotlib(self, enable: bool = True) -> None:
166      """Set up handlers for matplotlib's unit support.
167
168      Parameters
169      -----
170      enable : bool
171          whether support should be enabled or disabled (Default value = True)
172
173      """
```



`setup_matplotlib(enable: bool = True) -> None` [\[source\]](#)

Set up handlers for matplotlib's unit support.

Parameters:

`enable` ([bool](#)) – whether support should be enabled or disabled (Default value = True)



AGH



automating docs deployment with pdoc, GH Actions & Pages

```
name: pdoc
on:
  push:
    branches: [ main ]
jobs:
  pdoc:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install pdoc; python -m pdoc -o html mc_pi
      - uses: JamesIves/github-pages-deploy-action@v4
        with:
          folder: html
```


API Documentation

pi()

built with 

mc_pi

[► View Source](#)

```
def pi(n):
```

[▼ View Source](#)

```
3 pi = lambda n: 4 * sum(abs(u01() + 1j * u01()) < 1 for _ in range(n)) / n
```

Estimate π using a Monte Carlo method.

n: number of trials.



AGH



plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- Definition of test-assignment and grading criteria



AGH





Search

Get Started

How-to guides

Reference guides

Explanation

Anatomy of a test

About fixtures

Good Integration Practices

pytest import mechanisms and
sys.path / PYTHONPATH

Typing in pytest

CI Pipelines

Flaky tests

Examples and customization
tricks

ABOUT THE PROJECT

Anatomy of a test

In the simplest terms, a test is meant to look at the result of a particular behavior, and make sure that result aligns with what you would expect. Behavior is not something that can be empirically measured, which is why writing tests can be challenging.

"Behavior" is the way in which some system **acts in response** to a particular situation and/or stimuli. But exactly *how* or *why* something is done is not quite as important as *what* was done.

You can think of a test as being broken down into four steps:

1. **Arrange**
2. **Act**
3. **Assert**
4. **Cleanup**

Arrange is where we prepare everything for our test. This means pretty much everything except for the "**act**". It's lining up the dominoes so that the **act** can do its thing in one, state-changing step. This can mean preparing objects, starting/killing services, entering records into a database, or even things like defining a URL to query, generating some credentials for a user that doesn't exist yet, or just waiting for some process to finish.

Act is the singular, state-changing action that kicks off the **behavior** we want to test. This behavior is what carries out the changing of the state of the system under test (SUT), and it's the resulting changed state that we can look at to make a judgement about the behavior. This typically takes the form of a function/method call.

Assert is where we look at that resulting state and check if it looks how we'd expect after the dust has settled. It's where we gather evidence to say the behavior does or does not align with what we expect. The `assert` in our test is where we take that measurement/observation and apply our judgement to it. If something should be green, we'd say `assert thing == "green"`.

a hello-world unit test

```
%%file test_pi.py
```

```
import random
```

```
import math
```

```
import mc_pi
```

```
def test_pi():
```

```
    # arrange
```

```
    random.seed(44)
```

```
    # act
```

```
    value = mc_pi.pi(n=10000)
```

```
    # assert
```

```
    assert abs(math.pi - value) / math.pi < .001
```



AGH



invoking pytest

```
!pytest --quiet test_pi.py
```

F

=====

```
def test_pi():  
    # arrange  
    random.seed(44)
```

```
    # act  
    value = mc_pi.pi(n=10000)
```

```
    # assert
```

```
> assert abs(math.pi - value) / math.pi < .001
```

```
E assert (0.006792653589793307 / 3.141592653589793) < 0.001
```



AGH



13/23

using GH Actions for a Continuous-Integration (CI) setup

```
name: pytest

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install pytest
      - run: pytest
```



AGH



Summary

All jobs

test

Run details

Usage

Workflow file

Annotations

1 error and 1 warning

test

Started 10s ago

Search logs



- > ✓ Set up job 1s
- > ✓ Run actions/checkout@v4 1s
- > ✓ Run pip install pytest 3s
- ▼ ✗ Run pytest 1s

```
1 ▶ Run pytest
4 ===== test session starts =====
5 platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
6 rootdir: /home/runner/work/mc_pi/mc_pi
7 configfile: pyproject.toml
8 collected 1 item
9
10 test_pi.py F [100%]
11
12 ===== FAILURES =====
13 _____ test_pi _____
14
15     def test_pi():
16         # arrange
17         random.seed(44)
18
19         # act
20         value = mc_pi.pi(n=10000)
21
22         # assert
23 > assert abs(math.pi - value) / math.pi < .001
24 E assert (0.006792653589793307 / 3.141592653589793) < 0.001
```



15/23

„tests turn assumptions into guarantees’’

- ▶ executable specification
- ▶ regression safety net
- ▶ automated validation
- ▶ progress indicator
- ▶ documentation & onboarding aid
- ▶ design feedback (modularity)



AGH



plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- **Python projects: (test.)PyPI.org uploads with GH Actions**
- Definition of test-assignment and grading criteria



AGH



⚠ You are using TestPyPI – a separate instance of the Python Package Index that allows you to try distribution tools and processes without affecting the real index.



slayoo ▾

Test Python package publishing with the Test Python Package Index

Type '/' to search projects



Or [browse projects](#)

259,023 projects

1,562,349 releases

3,483,675 files

237,573 users



AGH



18/23

```
on:
  release:
    types: [published]
jobs:
  publish:
    permissions:
      id-token: write
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install build; python -m build
      - uses: pypa/gh-action-pypi-publish@release/v1.13
        with:
          repository_url: https://test.pypi.org/legacy/
          attestations: false
```



AGH



plan for today

- Python projects: generating documentation (from docstrings)
- Python projects: unit testing with pytest & GH Actions
- Python projects: (test.)PyPI.org uploads with GH Actions
- **Definition of test-assignment and grading criteria**



AGH



20/23

Test assignment (lecture material)

Create a (hello-world) public Python project on GitHub with:

- ▶ usage of either Numba or Pint (or both)
- ▶ CI-automated unit test[s] (e.g., using pytest)
- ▶ pyproject.toml-based package metadata (incl. dependencies, license, version)
- ▶ automated docs deployment (e.g., to GitHub Pages)
- ▶ automated on-release uploads to test.pypi.org
- ▶ colab.google/mybinder.org-ready Jupyter notebook with a demo (incl. installation)

Some extra suggestions:

- ▶ use setuptools_scm for automated git-coupled versioning
- ▶ setup automated code-coverage using (e.g., using codecov)

send a colab.google or mybinder.org link via e-mail by May 10



AGH



21/23

take-home messages:

- ▶ **not automated** $\rightsquigarrow$ not reproducible $\rightsquigarrow$ **not science**
- ▶ **not automated** $\rightsquigarrow$ not scalable $\rightsquigarrow$ **not productizable**
- ▶ not unit-tested
 - $\rightsquigarrow$ no robust way to check if it still works
 - $\rightsquigarrow$ fear-driven development, unrefactorable code
 - $\rightsquigarrow$ **technical debt**
- ▶ versioned & packaged
 - $\rightsquigarrow$ unambiguous & unassisted access/deployment

Thank you for your attention!

slides and other course resources at
<https://github.com/open-atmos-krk/rse-python-course>



AGH

